

Trường Đại học Công nghệ Thông tin – ĐHQG-HCM

Khoa Mạng máy tính & Truyền thông

Thực hành Giải thuật xử lý song song và phân bổ

Báo cáo thực hành giải thuật xử lý song song và phân bổ

Mã nhóm: MÃ NHÓM

Key: KEY

GVHD: Lê Minh Khánh Hội

Lê Phạm Hoàng Trung

Ngày nộp: Ngày 25 tháng 5 năm 2026

Thông tin thành viên

STT	Thành viên	Phân công chính
1	Họ tên – MSSV – nhiệm vụ chính	[CẦN ĐIỀN: challenge/phần việc]
2	Họ tên – MSSV – nhiệm vụ chính	[CẦN ĐIỀN: challenge/phần việc]
3	Họ tên – MSSV – nhiệm vụ chính	[CẦN ĐIỀN: challenge/phần việc]
4	Họ tên – MSSV – nhiệm vụ chính	[CẦN ĐIỀN: challenge/phần việc]

TP. Hồ Chí Minh, Ngày 25 tháng 5 năm 2026

Cam kết

Nhóm cam kết mã nguồn nộp lên hệ thống là phiên bản cuối cùng của nhóm, tuân thủ danh sách thư viện được phép dùng trong đề bài và không nộp mã nguồn tuần tự lên hệ thống chấm. Các số liệu hiệu năng trong báo cáo được đo từ môi trường thực nghiệm được mô tả tại Mục 1.

[CẦN ĐIỀN: Điền thêm cam kết hoặc chữ ký nếu giảng viên yêu cầu.]

Mục lục

1	Tổng quan yêu cầu và phương pháp đánh giá	5
1.1	Mục tiêu	5
1.2	Yêu cầu bắt buộc trong báo cáo	5
1.3	Môi trường thực nghiệm	5
1.4	Cách đo và công thức	5
1.5	Tóm tắt mã nguồn hiện có	6
2	Challenge 1: Tính tổng mảng số nguyên	6
2.1	Cách hiện thực trong mã nguồn	6
2.2	Phân tích song song hóa	6
2.3	Thiết kế thuật toán song song	7
2.4	Dữ liệu, phụ thuộc và chi phí	7
2.5	Đánh giá hiệu năng	7
3	Challenge 2: Số Fibonacci modulo	8
3.1	Cách hiện thực trong mã nguồn	8
3.2	Phân tích song song hóa	8
3.3	Thiết kế thuật toán song song	8
3.4	Dữ liệu, phụ thuộc và chi phí	9
3.5	Đánh giá hiệu năng	9
4	Challenge 3: Nhân hai ma trận	9
4.1	Cách hiện thực trong mã nguồn	9
4.2	Phân tích song song hóa	10
4.3	Thiết kế thuật toán song song	10
4.4	Dữ liệu, phụ thuộc và chi phí	10
4.5	Đánh giá hiệu năng	11
5	Challenge 4: Sắp xếp mảng một chiều	11
5.1	Cách hiện thực trong mã nguồn	11
5.2	Phân tích song song hóa	11
5.3	Thiết kế thuật toán song song	12
5.4	Tính ổn định	12
5.5	Dữ liệu, phụ thuộc và chi phí	12
5.6	Đánh giá hiệu năng	12
6	Challenge 5: Tìm kiếm trong mảng số nguyên lớn	13
6.1	Hiện trạng mã nguồn	13
6.2	Thiết kế đề xuất	13
6.3	Thiết kế thuật toán song song	13
6.4	Dữ liệu, phụ thuộc và chi phí	13
6.5	Đánh giá hiệu năng	14
7	Challenge 6: Phân tích không gian nhúng	14
7.1	Cách hiện thực trong mã nguồn	14
7.2	Phân tích song song hóa	14
7.3	Thiết kế thuật toán song song	15

7.4	Dữ liệu, phụ thuộc và chi phí	15
7.5	Đánh giá hiệu năng	15
8	Kết luận	16
8.1	Tổng hợp kết quả	16
8.2	Checklist trước khi nộp	16
A	Phụ lục: Mẫu đo hiệu năng	16
B	Phụ lục: Mẫu phiên bản tuần tự tham chiếu	17
C	Phụ lục: Checklist nội dung theo đề	17

Danh sách bảng

1	Thông tin môi trường đo hiệu năng	5
2	Tóm tắt các file challenge	6
3	Kết quả đo hiệu năng cho Challenge 1	7
4	Kết quả đo hiệu năng cho Challenge 2	9
5	Kết quả đo hiệu năng cho Challenge 3	11
6	Kết quả đo hiệu năng cho Challenge 4	12
7	Kết quả đo hiệu năng cho Challenge 5	14
8	Kết quả đo hiệu năng cho Challenge 6	15
9	Tổng hợp speedup tốt nhất sau khi đo	16
10	Phiên bản tuần tự tham chiếu đề xuất	17
11	Đối chiếu yêu cầu đề bài với báo cáo	17

Danh sách hình vẽ

1 Tổng quan yêu cầu và phương pháp đánh giá

1.1 Mục tiêu

Bài thực hành yêu cầu vận dụng lập trình song song để giải quyết sáu bài toán có độ phức tạp cao bằng Python. Mỗi hàm `MAIN(input_file_path)` phải đọc dữ liệu từ tệp, trả về đúng định dạng yêu cầu và tận dụng các thư viện được hệ thống chấm import sẵn như `multiprocessing`, `threading`, `concurrent.futures`, `joblib`, `math`, `collections`.

1.2 Yêu cầu bắt buộc trong báo cáo

Theo đề bài, với từng challenge, báo cáo cần trả lời đầy đủ ba nhóm nội dung:

- **Phân tích song song hóa:** phân việc có thể chạy song song, cách chia việc, cách phân phối/truy cập dữ liệu, phụ thuộc dữ liệu, cân bằng tải và chi phí giao tiếp.
- **Thiết kế thuật toán song song:** trình bày thuật toán chi tiết, có thể kèm pseudocode hoặc lưu đồ.
- **Đánh giá hiệu năng:** đo thời gian chương trình song song, so sánh với chương trình tuần tự tương ứng, tính speedup/efficiency và giải thích kết quả.

1.3 Môi trường thực nghiệm

Bảng 1: Thông tin môi trường đo hiệu năng

Hạng mục	Giá trị
CPU	[CẦN ĐIỀN: Tên CPU, số core/số thread]
RAM	[CẦN ĐIỀN: Dung lượng RAM]
Hệ điều hành	[CẦN ĐIỀN: OS và phiên bản]
Python	[CẦN ĐIỀN: Phiên bản Python trên máy đo hoặc server]
Số worker mặc định	[CẦN ĐIỀN: <code>multiprocessing.cpu_count()</code> hoặc cấu hình cụ thể]
Lệnh đo thời gian	[CẦN ĐIỀN: <code>time.perf_counter</code> hoặc công cụ đo]

1.4 Cách đo và công thức

Mỗi cấu hình nên chạy tối thiểu 3 lần, bỏ lần warm-up nếu cần, rồi lấy trung bình:

$$T = \frac{1}{k} \sum_{i=1}^k T_i.$$

Speedup và efficiency được tính theo:

$$S_p = T_{\text{seq}}/T_{\text{par}}, \quad E_p = S_p/p.$$

Trong đó T_{seq} là thời gian của phiên bản tuần tự tham chiếu, T_{par} là thời gian của phiên bản song song, và p là số worker.

1.5 Tóm tắt mã nguồn hiện có

Bảng 2: Tóm tắt các file challenge

Challenge	File	Hiện trạng chính
1	challenge/challenge1.py	Đọc toàn bộ byte, chia byte-range theo worker, dùng <code>multiprocessing.Pool.starmap</code> .
2	challenge/challenge2.py	Tính Fibonacci modulo bằng fast doubling hoặc bằng window, ghi kết quả vào <code>RawArray</code> .
3	challenge/challenge3.py	Nhân ma trận theo cụm dòng, lưu B^T , gom ma trận kết quả, hai đường chéo và tổng.
4	challenge/challenge4.py	Tự cài bottom-up merge sort, chia records thành chunks, sort song song rồi merge.
5	challenge/challenge5.py	File hiện tại rỗng; cần bổ sung code hoặc cập nhật phân tích sau khi có phiên bản cuối.
6	challenge/challenge6.py	Tính closest pair cho từng bộ dữ liệu bằng hash grid, song song hóa theo test case.

2 Challenge 1: Tính tổng mảng số nguyên

Yêu cầu bài toán

Đọc N số thực và chỉ cộng các giá trị đồng thời thỏa ba điều kiện: là số nguyên về mặt toán học, lớn hơn 0, và chia hết cho 3. Kết quả trả về là một số nguyên có thể vượt 32-bit.

2.1 Cách hiện thực trong mã nguồn

File `challenge/challenge1.py` đọc toàn bộ input ở dạng byte vào biến toàn cục `_global_data`. Sau khi bỏ dòng đầu chứa N , chương trình chia vùng dữ liệu còn lại thành nhiều đoạn byte. Mỗi worker gọi `_process_indices(start, end)` để split token trong đoạn của mình, ép kiểu float, kiểm tra `val.is_integer()`, điều kiện dương và chia hết cho 3, rồi trả về tổng cục bộ. Tiến trình chính cộng các tổng cục bộ bằng `sum(parts)`.

2.2 Phân tích song song hóa

Phép kiểm tra từng phần tử độc lập với mọi phần tử khác, vì vậy bài toán phù hợp với mô hình map-reduce:

- **Map:** mỗi worker xử lý một đoạn token và tính tổng cục bộ.
- **Reduce:** tiến trình chính cộng các tổng cục bộ.

Dữ liệu được chia theo byte-range thay vì theo số lượng token để tránh phải parse trước toàn bộ mảng. Để không cắt giữa một token, điểm cuối mỗi chunk được dịch tới khoảng trắng tiếp theo. Cách chia này giảm chi phí tiền xử lý nhưng có thể gây lệch tải nhẹ nếu các token có độ dài rất khác nhau.

2.3 Thiết kế thuật toán song song

Listing 1: Pseudocode song song cho Challenge 1

```

1 MAIN(path):
2     raw = read_all_bytes(path)
3     data_start = position_after_first_line(raw)
4     workers = choose_worker_count(len(raw) - data_start)
5     ranges = split_byte_ranges_at_token_boundary(raw, data_start,
6         workers)
7
8     partial_sums = pool.starmap(process_range, ranges)
9     return sum(partial_sums)
10
11 process_range(start, end):
12     total = 0
13     for token in raw[start:end].split():
14         value = float(token)
15         if value > 0 and value.is_integer() and int(value) % 3 == 0:
16             total += int(value)
17     return total

```

2.4 Dữ liệu, phụ thuộc và chi phí

- **Phân phối dữ liệu:** input được giữ trong biến toàn cục; worker chỉ nhận cặp chỉ số (start, end).
- **Phụ thuộc dữ liệu:** không có phụ thuộc giữa các phần tử; chỉ có bước reduce cuối.
- **Cân bằng tải:** gần cân bằng theo số byte; có thể kém cân bằng nếu token dài/ngắn không đều.
- **Chi phí giao tiếp:** thấp vì worker trả về một số nguyên cục bộ; chi phí chính là tạo process, split token và parse float.

2.5 Đánh giá hiệu năng

Bảng 3: Kết quả đo hiệu năng cho Challenge 1

Kích thước	Số worker	T_{seq} (s)	T_{par} (s)	Speedup
[CẦN ĐIỀN: size]	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]
[CẦN ĐIỀN: size]	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]
[CẦN ĐIỀN: size]	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]

Nhận xét. [CẦN ĐIỀN: So sánh thời gian tuần tự và song song. Giải thích khi input nhỏ có thể chậm hơn do overhead tạo process; input lớn thường có lợi vì parse và kiểm tra token độc lập.]

3 Challenge 2: Số Fibonacci modulo

Yêu cầu bài toán

Với N truy vấn A_i và modulo cố định Q , trả về danh sách $F(A_i) \bmod Q$ theo đúng thứ tự input. Ràng buộc lớn nhất là $N \leq 10^6$, $Q \leq 10^9$, $A_i \leq 2 \times 10^8$.

3.1 Cách hiện thực trong mã nguồn

File `challenge/challenge2.py` dùng hai chiến lược:

- `_fib_pair`: fast doubling, tính $F(n)$ trong $\mathcal{O}(\log n)$.
- `_build_tables`: dựng bảng window để tách n thành phần thấp/cao, từ đó tính nhanh $F(n)$ modulo Q bằng công thức cộng Fibonacci.

Kết quả được ghi vào `multiprocessing.RawArray` để bảo toàn thứ tự output và giảm chi phí gom danh sách. Khi số truy vấn đủ lớn, chương trình chia input theo byte-range, mỗi worker xử lý một đoạn truy vấn và ghi vào vùng kết quả tương ứng.

3.2 Phân tích song song hóa

Mỗi truy vấn Fibonacci độc lập với các truy vấn khác, nên có thể song song hóa theo tập truy vấn. Điểm cần chú ý là thứ tự output: worker xử lý chunk nào phải ghi kết quả vào đúng offset `out_start`. Bảng window là dữ liệu đọc chung sau khi dựng, phù hợp để chia sẻ qua cơ chế fork trên Linux.

3.3 Thiết kế thuật toán song song

Listing 2: Pseudocode song song cho Challenge 2

```

1 MAIN(path):
2     raw = read_all_bytes(path)
3     N, modulo, query_start = parse_header(raw)
4     if modulo == 1:
5         return [0] * N
6
7     build_window_tables_if_beneficial()
8     output = RawArray(unsigned_int_or_long_long, N)
9     chunks = split_queries_by_byte_range(raw, query_start, workers)
10
11     for each chunk in parallel:
12         tokens = raw[chunk.byte_start:chunk.byte_end].split()
13         for local_index, token in enumerate(tokens):
14             n = int(token)
15             output[chunk.out_start + local_index] = fib_mod(n, modulo)
16
17     return output[:]
```

3.4 Dữ liệu, phụ thuộc và chi phí

- **Phân phối dữ liệu:** input byte và bảng Fibonacci là dữ liệu đọc chung; output là RawArray ghi theo vùng không giao nhau.
- **Phụ thuộc dữ liệu:** không có phụ thuộc giữa các truy vấn; phụ thuộc duy nhất là bảng window phải dựng xong trước khi worker chạy.
- **Cân bằng tải:** chia theo byte và số dòng; nếu các truy vấn có số chữ số tương tự nhau thì tải tương đối đều.
- **Chi phí giao tiếp:** thấp vì worker không trả về danh sách lớn; chi phí nằm ở dựng bảng, parse token và đồng bộ process.

3.5 Đánh giá hiệu năng

Bảng 4: Kết quả đo hiệu năng cho Challenge 2

Kích thước	Số worker	T_{seq} (s)	T_{par} (s)	Speedup
[CẦN ĐIỀN: size]	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]
[CẦN ĐIỀN: size]	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]
[CẦN ĐIỀN: size]	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]

Nhận xét. [CẦN ĐIỀN: Phân tích lợi ích của bảng window so với fast doubling thuần. Nêu trường hợp modulo/size nhỏ khiến overhead dựng bảng và tạo process lớn hơn lợi ích.]

4 Challenge 3: Nhân hai ma trận

Yêu cầu bài toán

Cho hai ma trận vuông A và B kích thước $N \times N$, tính $C = A \times B$, đồng thời trả về đường chéo chính, đường chéo phụ và tổng toàn bộ phần tử của C .

4.1 Cách hiện thực trong mã nguồn

File challenge/challenge3.py đọc số nguyên bằng generator `_int_stream` để giảm chi phí split toàn bộ chuỗi. Ma trận B được lưu ở dạng chuyển vị `_BT_DATA`; nhờ đó tích vô hướng giữa một dòng của A và một cột của B trở thành truy cập tuần tự giữa hai list. Hàm `_mul_chunk(start_row, end_row)` tính các dòng $C[start_row : end_row]$, đồng thời trả về đoạn đường chéo chính, đường chéo phụ và tổng cục bộ.

4.2 Phân tích song song hóa

Mỗi dòng của ma trận kết quả C có thể tính độc lập sau khi đã có A và B^T . Vì vậy chiến lược chia việc tự nhiên là chia theo cụm dòng:

- Worker nhận một khoảng dòng $[s, e)$.
- Worker đọc toàn bộ A và B^T nhưng chỉ ghi/trả về các dòng C thuộc khoảng của mình.
- Tiến trình chính ghép các dòng theo `start_row`, ghép đường chéo và cộng tổng cục bộ.

4.3 Thiết kế thuật toán song song

Listing 3: Pseudocode song song cho Challenge 3

```

1 MAIN(path):
2     N = read_N(path)
3     A = read_matrix_A(path, N)
4     BT = read_transposed_matrix_B(path, N)
5     tasks = split_rows_into_chunks(N, workers, chunks_per_worker=4)
6
7     for each (start_row, end_row) in parallel:
8         rows = []
9         diag_main_part = []
10        diag_secondary_part = []
11        local_sum = 0
12
13        for i in range(start_row, end_row):
14            for j in range(N):
15                C[i][j] = dot(A[i], BT[j])
16                update diagonals and local_sum
17
18        merge rows, diagonals, and sums
19    return {matrix, diag_main, diag_secondary, total_sum}

```

4.4 Dữ liệu, phụ thuộc và chi phí

- **Phân phối dữ liệu:** A và B^T là dữ liệu đọc chung qua biến toàn cục trong worker; kết quả được trả về theo chunk.
- **Phụ thuộc dữ liệu:** các dòng C_i độc lập; không có worker nào cần kết quả của worker khác.
- **Cân bằng tải:** chia theo dòng cho tải gần đều vì mỗi dòng cần N^2 phép nhân-cộng.
- **Chi phí giao tiếp:** cao hơn các challenge khác vì worker trả về nhiều dòng của ma trận C ; với N lớn, chi phí bộ nhớ và pickle/IPC có thể chi phối.
- **Giới hạn bộ nhớ:** việc trả về toàn bộ C có độ phức tạp bộ nhớ $\mathcal{O}(N^2)$; khi thử $N = 5000$ cần ghi rõ cấu hình RAM và khả năng chạy thực tế.

4.5 Đánh giá hiệu năng

Bảng 5: Kết quả đo hiệu năng cho Challenge 3

N	Số worker	T_{seq} (s)	T_{par} (s)	Speedup	
128	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: p]
256	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: p]
512	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: p]
1024	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: p]
5000	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: p]

Nhận xét. [CẦN ĐIỀN: Giải thích xu hướng khi N tăng: lượng tính toán tăng nhanh nên song song hóa theo dòng có lợi hơn, nhưng chi phí bộ nhớ và trả ma trận kết quả có thể giới hạn speedup.]

5 Challenge 4: Sắp xếp mảng một chiều

Yêu cầu bài toán

Sắp xếp ổn định N bản ghi (v, label) theo v giảm dần, nếu trùng v thì label tăng dần từ điển, nếu cả hai trùng thì giữ thứ tự ban đầu. Không dùng hàm sắp xếp dựng sẵn của Python.

5.1 Cách hiện thực trong mã nguồn

File `challenge/challenge4.py` biểu diễn mỗi bản ghi thành tuple $(-v, \text{label})$ để dùng thứ tự tăng dần của tuple thay cho yêu cầu v giảm dần. Thuật toán sort cục bộ là bottom-up merge sort tự cài trong `bottom_up_merge_sort`. Với input lớn, `parallel_sort` chia records thành nhiều đoạn, sort từng đoạn bằng `multiprocessing.Pool.map`, sau đó merge từng cặp đoạn bằng `merge_two` cho tới khi còn một danh sách.

5.2 Phân tích song song hóa

Merge sort có hai mức có thể song song:

- Sort các chunk ban đầu độc lập.
- Merge các cặp chunk theo từng tầng.

Mã hiện tại song song hóa giai đoạn sort chunk, còn giai đoạn merge được thực hiện trong tiến trình chính. Đây là lựa chọn đơn giản, giảm chi phí trao đổi dữ liệu giữa process nhưng có thể khiến merge cuối trở thành bottleneck khi N rất lớn.

5.3 Thiết kế thuật toán song song

Listing 4: Pseudocode song song cho Challenge 4

```

1 MAIN(path):
2     records = read_records(path)
3     transformed = [(-value, label) for each record]
4     sorted_records = parallel_sort(transformed)
5     return [(-value, label.decode()) for each item in sorted_records]
6
7 parallel_sort(records):
8     if len(records) < threshold:
9         return bottom_up_merge_sort(records)
10
11     ranges = split_array_ranges(records, workers)
12     chunks = pool.map(sort_range, ranges)
13     return merge_chunks(chunks)

```

5.4 Tính ổn định

Trong merge sort, khi hai phần tử bằng nhau, hàm merge chọn phần tử bên trái trước vì điều kiện $a \leq b$. Điều này giữ thứ tự trong từng chunk. Khi merge các chunk theo thứ tự ban đầu, các phần tử bằng nhau ở chunk bên trái cũng được giữ trước chunk bên phải, nên tính ổn định được bảo toàn.

5.5 Dữ liệu, phụ thuộc và chi phí

- **Phân phối dữ liệu:** mảng records được chia thành các range; worker đọc slice tương ứng từ biến toàn cục.
- **Phụ thuộc dữ liệu:** sort cục bộ độc lập; merge phụ thuộc vào kết quả sort của các chunk.
- **Cân bằng tải:** chia theo số lượng record nên cân bằng tương đối đều.
- **Chi phí giao tiếp:** đáng kể vì mỗi worker trả về một danh sách đã sort; merge tuần tự cũng tiêu tốn $\mathcal{O}(N \log p)$.

5.6 Đánh giá hiệu năng

Bảng 6: Kết quả đo hiệu năng cho Challenge 4

N	Số worker	T_{seq} (s)	T_{par} (s)	Speedup	Eff
10^4	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: eff]
10^5	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: eff]
10^6	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: eff]

Nhận xét. [CẦN ĐIỀN: Nêu rõ ngưỡng N nhỏ có thể chạy tuần tự nhanh hơn do overhead; với N lớn, sort chunk song song có lợi nhưng merge tuần tự và chi phí truyền list làm speedup không tuyến tính.]

6 Challenge 5: Tìm kiếm trong mảng số nguyên lớn

Yêu cầu bài toán

Input gồm khóa K và một mảng số nguyên lớn tối đa 10^6 phần tử. Cần tìm chỉ số nhỏ nhất i sao cho $A[i] = K$, nếu không có thì trả về -1 . Không dùng hàm tìm kiếm dựng sẵn; cần chia mảng thành nhiều đoạn và phối hợp worker.

6.1 Hiện trạng mã nguồn

File `challenge/challenge5.py` hiện đang rỗng tại thời điểm lập báo cáo. Vì vậy section này được viết theo thiết kế đề xuất từ yêu cầu bài toán. Sau khi nhóm bổ sung phiên bản cuối của Challenge 5, cần cập nhật lại phần “Cách hiện thực trong mã nguồn” để khớp chính xác với code.

6.2 Thiết kế đề xuất

Chiến lược song song hóa phù hợp là chia mảng A thành các đoạn liên tiếp. Mỗi worker tìm vị trí đầu tiên của K trong đoạn của mình và trả về chỉ số toàn cục nhỏ nhất trong đoạn, hoặc -1 nếu không tìm thấy. Tiềm năng chính lấy min trên các kết quả khác -1 .

6.3 Thiết kế thuật toán song song

Listing 5: Pseudocode đề xuất cho Challenge 5

```
1 MAIN(path):
2     K, A = parse_input(path)
3     ranges = split_index_ranges(len(A), workers)
4
5     candidates = pool.map(search_range, ranges)
6     valid = [idx for idx in candidates if idx != -1]
7     if valid is empty:
8         return -1
9     return min(valid)
10
11 search_range(start, end):
12     for i in range(start, end):
13         if A[i] == K:
14             return i
15     return -1
```

6.4 Dữ liệu, phụ thuộc và chi phí

- **Phân phối dữ liệu:** mảng A có thể đặt trong biến toàn cục để worker chỉ nhận khoảng chỉ số.
- **Phụ thuộc dữ liệu:** mỗi đoạn tìm kiếm độc lập; bước reduce chỉ chọn chỉ số nhỏ nhất.

- **Cân bằng tải:** nếu không có cơ chế dừng sớm, mỗi worker quét gần như số phần tử bằng nhau. Nếu có dừng sớm, cần đồng bộ cân thận để vẫn đảm bảo chỉ số nhỏ nhất.
- **Chi phí giao tiếp:** thấp vì mỗi worker chỉ trả về một chỉ số.

6.5 Đánh giá hiệu năng

Bảng 7: Kết quả đo hiệu năng cho Challenge 5

Kích thước	Số worker	T_{seq} (s)	T_{par} (s)	Speedup
[CẦN ĐIỀN: size]	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]
[CẦN ĐIỀN: size]	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]
[CẦN ĐIỀN: size]	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]

Nhận xét. [CẦN ĐIỀN: Sau khi có code cuối, điền số liệu cho các trường hợp K xuất hiện đầu mảng, giữa mảng, cuối mảng và không xuất hiện. So sánh ảnh hưởng của dừng sớm nếu có.]

7 Challenge 6: Phân tích không gian nhúng

Yêu cầu bài toán

Với Q bộ dữ liệu điểm 2D, tìm khoảng cách Euclid nhỏ nhất giữa hai điểm bất kỳ trong từng bộ và trả về danh sách kết quả làm tròn 4 chữ số thập phân.

7.1 Cách hiện thực trong mã nguồn

File `challenge/challenge6.py` đọc toàn bộ token số nguyên, xác định range của từng test case rồi lưu vào `_G_RANGES`. Với mỗi bộ dữ liệu, `_solve_one_raw(case_idx)` mã hóa điểm (x, y) thành một số nguyên 64-bit sau khi cộng offset để xử lý tọa độ âm. Nếu phát hiện điểm trùng, kết quả là 0.0. Nếu không, `_closest_pair_core` dùng kỹ thuật hash grid: lấy một số điểm đầu làm mẫu để ước lượng khoảng cách tốt nhất ban đầu, sau đó đưa điểm vào lưới kích thước d và chỉ so sánh với các ô lân cận.

7.2 Phân tích song song hóa

Các bộ dữ liệu độc lập với nhau, nên mã hiện tại song song hóa theo test case:

- Với $Q = 1$ hoặc tổng số điểm nhỏ, chạy tuần tự để tránh overhead process.
- Với nhiều bộ dữ liệu lớn, sắp xếp thứ tự xử lý theo N giảm dần để worker nhận case lớn trước, giảm nguy cơ worker cuối bị giữ bởi một case rất lớn.
- Worker trả về cặp $(case_idx, ans)$ để tiến trình chính đặt kết quả về đúng thứ tự ban đầu.

7.3 Thiết kế thuật toán song song

Listing 6: Pseudocode song song cho Challenge 6

```

1 MAIN(path):
2     tokens = parse_all_ints(path)
3     ranges = locate_each_case(tokens)
4
5     if small_input:
6         return [solve_case(i) for i in range(Q)]
7
8     order = cases_sorted_by_size_descending()
9     answers = [0.0] * Q
10    for case_idx, ans in pool.imap_unordered(solve_case, order):
11        answers[case_idx] = ans
12    return answers
13
14 solve_case(case_idx):
15     points = encode_points_as_int_set(case_idx)
16     if duplicate_exists:
17         return 0.0
18     return closest_pair_hash_grid(points)

```

7.4 Dữ liệu, phụ thuộc và chi phí

- **Phân phối dữ liệu:** token input và range từng case là dữ liệu đọc chung; worker nhận chỉ số case.
- **Phụ thuộc dữ liệu:** không có phụ thuộc giữa các case; trong một case, hash grid là cấu trúc cục bộ của worker.
- **Cân bằng tải:** sắp xếp case lớn trước giúp cân bằng tốt hơn khi kích thước các case chênh lệch.
- **Chi phí giao tiếp:** thấp vì worker chỉ trả về một số thực; chi phí chính nằm ở parse input, tạo set điểm và xử lý hash grid.
- **Độ đúng:** cần nhấn mạnh cơ chế so sánh trong các ô lân cận và cập nhật lại kích thước lưới khi tìm được khoảng cách nhỏ hơn.

7.5 Đánh giá hiệu năng

Bảng 8: Kết quả đo hiệu năng cho Challenge 6

Kích thước	Số worker	T_{seq} (s)	T_{par} (s)	Speedup
[CẦN ĐIỀN: size]	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]
[CẦN ĐIỀN: size]	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]
[CẦN ĐIỀN: size]	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]

Nhận xét. [CẦN ĐIỀN: Điền số liệu theo số case Q, tổng số điểm và phân bố kích thước case. Giải thích trường hợp Q nhỏ khó tận dụng song song theo case.]

8 Kết luận

Báo cáo đã trình bày phương pháp song song hóa cho sáu challenge theo yêu cầu đề bài. Nhìn chung, các bài toán có thể chia thành hai nhóm:

- **Song song dữ liệu trực tiếp:** Challenge 1, 2 và 5 xử lý từng phần tử/truy vấn độc lập, bước gom kết quả đơn giản.
- **Song song theo cấu trúc bài toán:** Challenge 3 chia theo dòng ma trận, Challenge 4 chia theo chunk của merge sort, Challenge 6 chia theo từng bộ dữ liệu điểm.

Các yếu tố ảnh hưởng lớn nhất đến hiệu năng là chi phí tạo process, kích thước dữ liệu truyền giữa worker và tiến trình chính, cân bằng tải giữa các worker, và giới hạn bộ nhớ khi phải trả về kết quả lớn như ma trận C trong Challenge 3.

8.1 Tổng hợp kết quả

Bảng 9: Tổng hợp speedup tốt nhất sau khi đo

Challenge	Chiến lược chính	Kích thước tốt nhất	Speedup tốt nhất	Nhận xét
1	Chia byte-range, reduce tổng	[CẦN ĐIỀN: size]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN]
2	Chia truy vấn, RawArray	[CẦN ĐIỀN: size]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN]
3	Chia theo dòng ma trận	[CẦN ĐIỀN: size]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN]
4	Sort chunk song song, merge tuần tự	[CẦN ĐIỀN: size]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN]
5	Chia đoạn tìm kiếm	[CẦN ĐIỀN: size]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN]
6	Chia theo test case	[CẦN ĐIỀN: size]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN]

8.2 Checklist trước khi nộp

- Đã điền mã nhóm, key và thông tin thành viên ở `main.tex`.
- Đã cập nhật Challenge 5 theo code cuối cùng nếu nhóm có nộp challenge này.
- Đã đo và điền đủ bảng thời gian tuần tự/song song cho từng challenge.
- Đã giải thích các trường hợp speedup thấp hoặc chương trình song song chậm hơn tuần tự.
- Đã build thành PDF và kiểm tra mục lục, bảng, hình, số trang.
- Đã bảo đảm mã nguồn tuần tự chỉ dùng để benchmark, không nộp lên hệ thống chấm.

A Phụ lục: Mẫu đo hiệu năng

Phần này là gợi ý cách tổ chức đo thời gian. Nhóm có thể thay bằng script thực tế đã dùng.

Listing 7: Mẫu đo thời gian một hàm MAIN

```

1 from time import perf_counter
2
3 def measure(func, path, repeat=3):
4     times = []
5     last_result = None
6     for _ in range(repeat):
7         start = perf_counter()
8         last_result = func(path)
9         end = perf_counter()
10        times.append(end - start)
11    return sum(times) / len(times), last_result

```

B Phụ lục: Mẫu phiên bản tuần tự tham chiếu

Phiên bản tuần tự chỉ dùng để đo so sánh trong báo cáo, không nộp lên hệ thống chấm. Khi trình bày trong báo cáo, nên mô tả ngắn gọn phiên bản tuần tự tương ứng thay vì đưa toàn bộ mã nguồn nếu quá dài.

Bảng 10: Phiên bản tuần tự tham chiếu đề xuất

Challenge	Thuật toán tuần tự	Ghi chú
1	Quét token một lượt, cộng trực tiếp	Cùng điều kiện lọc với bản song song.
2	Fast doubling cho từng truy vấn	Có thể dùng cùng hàm <code>_fib_pair</code> .
3	Ba vòng lặp nhân ma trận	Có thể vẫn dùng B^T để so sánh công bằng truy cập bộ nhớ.
4	Bottom-up merge sort một luồng	Không dùng <code>sorted</code> hoặc <code>list.sort</code> .
5	Quét tuyến tính từ trái sang phải	Trả về ngay khi gặp K .
6	Thuật toán cùng hash grid chạy tuần tự từng case	Hoặc chia để trị tuần tự nếu nhóm hiện thực

C Phụ lục: Checklist nội dung theo đề

Bảng 11: Đối chiếu yêu cầu đề bài với báo cáo

Yêu cầu	Vị trí trong báo cáo	Trạng thái
Phân tích phần song song hóa	Mỗi section challenge, tiểu mục “Phân tích song song hóa”	[CẦN ĐIỀN: OK]
Phương pháp chia việc	Mỗi section challenge, tiểu mục phân tích và thuật toán	[CẦN ĐIỀN: OK]
Phân phối và truy cập dữ liệu	Mỗi section challenge, tiểu mục “Dữ liệu, phụ thuộc và chi phí”	[CẦN ĐIỀN: OK]

Yêu cầu	Vị trí trong báo cáo	Trạng thái
Phân tích phụ thuộc dữ liệu, cân bằng tải, chi phí giao tiếp	Mỗi section challenge	[CẦN ĐIỀN: OK]
Thuật toán song song chi tiết	Pseudocode trong từng challenge	[CẦN ĐIỀN: OK]
Sơ đồ/lưu đồ nếu cần	[CẦN ĐIỀN: Chèn hình hoặc TikZ tại challenge cần minh họa]	[CẦN ĐIỀN: Cần kiểm tra]
Đo thời gian chương trình song song	Bảng hiệu năng từng challenge	[CẦN ĐIỀN: Cần điền]
So sánh với chương trình tuần tự	Bảng hiệu năng từng challenge	[CẦN ĐIỀN: Cần điền]
Giải thích kết quả	Đoạn “Nhận xét” từng challenge	[CẦN ĐIỀN: Cần điền]
Mã nhóm, key, thành viên	Trang bìa	[CẦN ĐIỀN: Cần điền]